REPORT

Reviewer report: pytoolz/toolz#658: Fix bytes and

bytearray source keyword signatures

Run toolz-658, tested 2026-11-09 14:37 IST by ops/daily_run.py (route source-pair).
Proof-of-Change commit

ef12096677e9cf59e8673436d74ec65d53771d7c. Report generated 2026-11-09 18:38 IST by

reports/pr_report.py.

Baseline: full (test comparison + claim probes + coverage gate + claim flagging (differential probes shown,

informational)).

VERIFIED

Recommended action: Approve with notes Confidence: Medium

The PR's tests behave the same or better than on main, in all 3 runs per side. Some checks gave warnings:

Install docs unchanged by the PR

Advice only: Proof-of-Change never blocks a merge and never posts on GitHub.

Summary in plain English

• What it says it does: Fix bytes and bytearray source keyword signatures (pytoolz/toolz#658).

• What we observed: cross-run settled differs claim(s) C1, C2 as proven (4 new or changed test(s) fail on the

base and pass on the PR:

toolz.tests.test_bytes_source_keyword::test_source_keyword_validation[is_partial_args-bytearray],

toolz.tests.test_bytes_source_keyword::test_source_keyword_validation[is_partial_args-bytes],

toolz.tests.test_bytes_source_keyword::test_source_keyword_validation[is_valid_args-bytearray] (+1 more)); 4

test(s) fail on main and pass on the PR; 7 test(s) ran on main, 9 on the PR; 2 new test(s) pass, 3 unchanged.

• What you should do: You can approve; read the notes under 'What went wrong' first.

Demo helper

These notes are background from the console and the daily check. They do not change the verdict.

If setup fails, the helper retries the install up to 2 times. If a test still can't be verified because something is

missing or fails to import, it looks up the cause, installs the fix on both sides and reruns that step, up to 3 tries.

• Setup check: GitHub token: present.

• Setup check: Nebius token: present.

• Setup check: free memory 3.3 GB (of 15.6 GB).

• Setup check: free disk 92.5 GB.

• Demo helper: git is installed.

• Demo helper: python3 is installed.

• Demo helper: uv is installed.

• Looked up with Tavily: [redacted] results

• Query: toolz python setup install instructions https://github.com/pytoolz/toolz site:github.com/pytoolz/toolz

• Query: toolz python documentation install https://github.com/pytoolz/toolz site:github.com/pytoolz/toolz

• Query: toolz python install requirements https://pypi.org/project/toolz/ https://github.com/pytoolz/toolz

site:pypi.org

• Source: toolz/pyproject.toml at master · pytoolz/toolz · GitHub —

http://github.com/pytoolz/toolz/blob/master/pyproject.toml

• Source: toolz/pyproject.toml at master · pytoolz/toolz · GitHub —

https://github.com/pytoolz/toolz/blob/master/pyproject.toml

• Source: toolz/README.rst at master · pytoolz/toolz · GitHub —

https://github.com/pytoolz/toolz/blob/master/README.rst

• Source: toolz/README.rst at master · pytoolz/toolz · GitHub —

http://github.com/pytoolz/toolz/blob/master/README.rst

• Source: toolz — https://pypi.org/project/toolz/

• Source: toolz/.github/workflows/test.yml at 451af60dec590a6010e2babdbf391ea8f815122f · pytoolz/toolz ·

GitHub —

https://github.com/pytoolz/toolz/blob/451af60dec590a6010e2babdbf391ea8f815122f/.github/workflows/test.yml

• Source: GitHub - pytoolz/toolz: A functional standard library for Python. · GitHub —

https://github.com/pytoolz/toolz

• Setup plan: the PR changes install docs (.github/workflows/test.yml); setup was read from main's copy and

applied the same on both sides.

• Setup plan: the same packages on main and pr.

• Setup plan: install coverage (listed in the test extra) on main and pr.

• Setup plan: install pycodestyle (listed in the test extra) on main and pr.

• Setup plan: install pytest (listed in the test extra) on main and pr.

• Setup plan: install sphinx (listed in requirements.txt) on main and pr.

• Setup plan: install furo (listed in requirements.txt) on main and pr.

• Setup plan: Python 3.12: Python 3.12 already meets >=3.9

• Setup plan: refused supports: no network or the package was not found

• Setup plan: refused Python: no network or the package was not found

• Setup plan: refused with: its PyPI page does not link to this GitHub repository

• Setup plan: refused a: its PyPI page does not link to this GitHub repository

• Setup plan: refused common: its PyPI page does not link to this GitHub repository

• Setup plan: refused codebase.: no network or the package was not found

• Setup plan: refused It: its PyPI page does not link to this GitHub repository

• Installed coverage 7.17.2 from pyproject (listed in the test extra) on main and pr.

• Installed pycodestyle 2.15.0 from pyproject (listed in the test extra) on main and pr.

• Installed pytest 9.2.0 from pyproject (listed in the test extra) on main and pr.

• Installed sphinx 9.1.1 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed furo 2025.12.19 from requirements.txt (listed in requirements.txt) on main and pr.

Claims vs reality

| # | Claim (source) | Status | What we observed | Log line

| C2 | Existing behavior is kept: tests that pass on main still pass (implied by every PR) | proven | no test that

passes on main fails on the PR (3 unchanged, 3 runs per side) | pytest-pr-1.log:7 «9 passed in 0.69s»

| P-C1 | bytes and bytearray functions now accept source keyword arguments. (PR description) | not observed |

probe model-probe-c1-bytes-source, c1.bytes.source_keyword_valid: same on both sides: false |

probe-model-probe-c1-bytes-source-pr-1.jsonl:1 «"observation": "c1.bytes.source_keyword_valid"»

| P-C1 | bytes and bytearray functions now accept source keyword arguments. (PR description) | not observed |

probe model-probe-c1-bytearray-source, c1.bytearray.source_keyword_valid: same on both sides: false |

probe-model-probe-c1-bytearray-source-pr-1.jsonl:1 «"observation": "c1.bytearray.source_keyword_valid"»

| P-C2 | bytes and bytearray functions now require encoding when converting strings. (PR description) | not

observed | probe model-probe-c2-bytes-string-no-encoding, c2.bytes.string_no_encoding_valid: same on both

sides: true | probe-model-probe-c2-bytes-string-no-encoding-pr-1.jsonl:1 «"observation":

"c2.bytes.string_no_encoding_valid"»

| P-C2 | bytes and bytearray functions now require encoding when converting strings. (PR description) | not

observed | probe model-probe-c2-bytearray-string-no-encoding, c2.bytearray.string_no_encoding_valid: same

on both sides: true | probe-model-probe-c2-bytearray-string-no-encoding-pr-1.jsonl:1 «"observation":

"c2.bytearray.string_no_encoding_valid"»

Citation check (PASS): 5 citation(s), every one re-read from its log file and matched.

What changed in behavior

| What | Count | Examples

| Tests that actually ran (main / PR, fewest of the runs) | 7 / 9 | | Tests unchanged | 3 | | New tests that pass | 2 | | Tests that pass on main but fail on the PR | 0 | | Tests that are new in the PR and fail | 0 | | Tests that ran on main but are missing on the PR side | 0 | | Tests that changed result in another way (for example passed -> skipped) | 0 | | Tests that gave different results between the 3 runs of the same side (flaky) | 0 | | Tests that fail on main and pass on the PR | 4 |

toolz.tests.test_bytes_source_keyword::test_source_keyword_validation[is_partial_args-bytearray],

toolz.tests.test_bytes_source_keyword::test_source_keyword_validation[is_partial_args-bytes],

toolz.tests.test_bytes_source_keyword::test_source_keyword_validation[is_valid_args-bytearray],

toolz.tests.test_bytes_source_keyword::test_source_keyword_validation[is_valid_args-bytes]

| Tests that are new or changed but did not run because an optional module is missing | 0 | | Tests that were re-parametrized (same test, new parameter ids; not counted as removed) | 0 | | Test modules that fail to load on the PR side only (their tests no longer run) | 0 | | Test modules that fail to load on main only (no baseline for their tests) | 0 | | Test modules that fail to load on both sides (their tests did not run) | 0 | | Tests that fail identically on both sides (already broken on main) | 0 | -

Probes

Model-written probes: model provider: nebius (Token Factory, live).

A probe is a small deterministic script run on both sides in the same environments as the tests, 3 alternating runs

per side. It prints JSON observations; plain code (poc/diff.py) compares them. A difference that no claim of the

PR covers needs attention. Every probe was frozen (content sha256) before any PR-side code ran, and checked

against that freeze before each PR-side run. Only a live model probe is blind; fixed probes and recorded model

fixtures are known-answer cases.

model-probe-c1-bytes-source (model (nvidia/nemotron-3-super-120b-a12b, live),

model-probes/model-probe-c1-bytes-source.py, frozen adc05a71b995 at 2026-11-09T09:08:48.358461+00:00):

Test if bytes accepts source keyword argument

• P-C1 (PR description): bytes and bytearray functions now accept source keyword arguments. Expects the

covered observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c1.bytes.source_keyword_valid | P-C1 | no change although the claim says it changes | same on both sides:

false | probe-model-probe-c1-bytes-source-main-1.jsonl:1 «"observation":

"c1.bytes.source_keyword_valid"»
probe-model-probe-c1-bytes-source-pr-1.jsonl:1 «"observation":

"c1.bytes.source_keyword_valid"»

model-probe-c1-bytearray-source (model (nvidia/nemotron-3-super-120b-a12b, live),

model-probes/model-probe-c1-bytearray-source.py, frozen 7ef06f78246c at

2026-11-09T09:08:48.358623+00:00): Test if bytearray accepts source keyword argument

• P-C1 (PR description): bytes and bytearray functions now accept source keyword arguments. Expects the

covered observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c1.bytearray.source_keyword_valid | P-C1 | no change although the claim says it changes | same on both

sides: false | probe-model-probe-c1-bytearray-source-main-1.jsonl:1 «"observation":

"c1.bytearray.source_keyword_valid"»
probe-model-probe-c1-bytearray-source-pr-1.jsonl:1 «"observation":

"c1.bytearray.source_keyword_valid"»

model-probe-c2-bytes-string-no-encoding (model (nvidia/nemotron-3-super-120b-a12b, live),

model-probes/model-probe-c2-bytes-string-no-encoding.py, frozen a909b0bfe7b8 at

2026-11-09T09:08:48.358699+00:00): Test if bytes requires encoding for string conversion (by omitting

encoding)

• P-C2 (PR description): bytes and bytearray functions now require encoding when converting strings. Expects

the covered observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c2.bytes.string_no_encoding_valid | P-C2 | no change although the claim says it changes | same on both sides:

true | probe-model-probe-c2-bytes-string-no-encoding-main-1.jsonl:1 «"observation":

"c2.bytes.string_no_encoding_valid"»
probe-model-probe-c2-bytes-string-no-encoding-pr-1.jsonl:1

«"observation": "c2.bytes.string_no_encoding_valid"»

model-probe-c2-bytearray-string-no-encoding (model (nvidia/nemotron-3-super-120b-a12b, live),

model-probes/model-probe-c2-bytearray-string-no-encoding.py, frozen d45f64b08646 at

2026-11-09T09:08:48.358809+00:00): Test if bytearray requires encoding for string conversion (by omitting

encoding)

• P-C2 (PR description): bytes and bytearray functions now require encoding when converting strings. Expects

the covered observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c2.bytearray.string_no_encoding_valid | P-C2 | no change although the claim says it changes | same on both

sides: true | probe-model-probe-c2-bytearray-string-no-encoding-main-1.jsonl:1 «"observation":

"c2.bytearray.string_no_encoding_valid"»
probe-model-probe-c2-bytearray-string-no-encoding-pr-1.jsonl:1

«"observation": "c2.bytearray.string_no_encoding_valid"»

Citation check (PASS): 8 citation(s), every one re-read from its log file and matched.

Was the changed code exercised?

Coverage: MEASURED by coverage.py on the PR side (1.9 s run).

Coverage gate (python, confidence normal): 2 of 2 changed executable line(s) ran (100%).

| Changed file | Changed executable lines | Of those, ran

| toolz/_signatures.py | 2 | 2

How this is measured: one extra, untimed run of the PR side's tests under coverage.py. Changed lines come

from comparing the tested main and PR files. Compiled code (C, C++, Cython) is invisible to coverage.py; such a

change only counts as verified when the PR adds or changes a test for that module.

What went right

• Tested 55b1b5e8744d of the PR against main (source-pair route).

• 3 alternating runs per side: main 3, PR 3 of 3.

• pytest exit codes explained: all runs exited 0.

• Changed code was exercised (coverage gate): 2 of 2 changed executable line(s) ran (100%).

• Probes ran on both sides: 4 probe(s), 3 alternating runs per side, every run printed its observations.

• Single-thread environment: OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0.

• Machine type recorded: local x86_64 8 cpus.

• Nothing posted upstream: GitHub was only read.

• Known-answer replay (fresh re-run): replayed 2026-10-10 06:30:21 IST, harness 1.0.

• Every claim cites a real log line: 5 citation(s), every one re-read from its log file and matched.

• Every probe observation cites its output line: 8 citation(s), every one re-read from its log file and matched.

What went wrong and how to fix it

| Issue | Status | Detail | How to fix

| Install docs unchanged by the PR | WARN | the PR changes .github/workflows/test.yml | Setup was read from

main's install docs and applied the same on both sides; the PR's own install steps differ, so read the result with

that in mind.

Evals and guardrails

| Check | Result | Detail

| 3 alternating runs per side | PASS | main 3, PR 3 of 3

| pytest exit codes explained | PASS | all runs exited 0

| Changed code was exercised (coverage gate) | PASS | 2 of 2 changed executable line(s) ran (100%)

| Probes ran on both sides | PASS | 4 probe(s), 3 alternating runs per side, every run printed its observations

| Install docs unchanged by the PR | WARN | the PR changes .github/workflows/test.yml

| Single-thread environment | PASS | OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0

| Machine type recorded | PASS | local x86_64 8 cpus

| Nothing posted upstream | PASS | GitHub was only read

| Known-answer replay (fresh re-run) | PASS | replayed 2026-10-10 06:30:21 IST, harness 1.0

| Every claim cites a real log line | PASS | 5 citation(s), every one re-read from its log file and matched

| Every probe observation cites its output line | PASS | 8 citation(s), every one re-read from its log file and

matched

Checklist

• I read the verdict and the plain-English summary.

• Every 'false' or 'unmentioned change' row has a log line I can open.

• Nothing under 'What went wrong' is a run problem (those mean Re-run, not Reject).

• I recorded my decision (see below).

Decision recorded: none yet. To record one, run make decide

REPORT=poc-main/reports/daily/2026-11-09/toolz-658 DECISION=approve (or reject,

approve_with_notes, re_run).

Rerun command

make check PR=https://github.com/pytoolz/toolz/pull/658

Evidence links

• Pull request pytoolz/toolz#658
• record.json
• pr.diff
• setup-main.log
• setup-pr.log
• git.log
• pytest-main-1.log
• pytest-pr-1.log
• pytest-main-2.log
• pytest-pr-2.log
• pytest-main-3.log
• pytest-pr-3.log
• junit-main-1.xml
• junit-pr-1.xml
• coverage-pytest.log
• coverage-pr.json
• probe-model-probe-c1-bytes-source-main-1.jsonl
• probe-model-probe-c1-bytes-source-pr-1.jsonl
• probe-model-probe-c1-bytearray-source-main-1.jsonl

• probe-model-probe-c1-bytearray-source-pr-1.jsonl
• probe-model-probe-c2-bytes-string-no-encoding-main-1.jsonl
• probe-model-probe-c2-bytes-string-no-encoding-pr-1.jsonl
• probe-model-probe-c2-bytearray-string-no-encoding-main-1.jsonl
• probe-model-probe-c2-bytearray-string-no-encoding-pr-1.jsonl
• main run 1 result: pytest-main-1.log:7 «3 passed in 0.59s»
• pr run 1 result: pytest-pr-1.log:7 «9 passed in 0.69s»

Glossary

| Term | Meaning
| main | The library as it is today, without the PR.
| PR | Pull request: a proposed change to the library.
| run | One full pass of the selected tests. We do 3 per side, alternating main and PR.
| flaky | A test that gives different results when nothing changed; it is ignored as evidence.

| log line | A line in a pytest-*.log file: the raw output the verdict is based on.

| route | How the two sides were built: nightly+patch, source-build or source-pair (see docs/OPERATIONS.md).

| changed lines covered | How many of the PR's changed, executable Python lines ran during the PR side's tests.

| re-parametrized | A test whose parameter ids changed (renamed cases); the new ids are judged on their own.

rebuilt from cleaned text

Written 2026-11-09 18:39 IST